Department of Computing

Lab 11

Concurrency Control: Suzuki-Kasami Algorithm

Name	: Abrar Butt
Qalam ID	: 460297
Course Code	: CS332
Course	: Distributed Computing
Class	: BSCS
Instructor	: Dr. Shah Khalid

April 16, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Tools / Software Requirements **2**

4 Background Theory **2**
- 4.1 Token-Based vs. Non-Token-Based Algorithms . . . 2
- 4.2 Data Structures . . . 2
- 4.3 Algorithm Steps . . . 3
  - 4.3.1 To Request / Enter the Critical Section . . . 3
  - 4.3.2 To Execute the Critical Section . . . 3
  - 4.3.3 To Release the Critical Section . . . 3

5 Implementation **3**
- 5.1 Complete Source Code — `suzuki_kasami.c` . . . 3

6 Compilation and Execution Guide **11**
- 6.1 Compile . . . 11
- 6.2 Run (5 Separate Terminals) . . . 11

7 Step-by-Step Execution Trace **11**

8 Expected Output (Screenshots) **12**
- 8.1 Terminal — Site 1 (Token Holder at Start) . . . 12
- 8.2 Terminal — Site 2 (Requests After Site 1) . . . 12
- 8.3 Terminal — Site 3 (Concurrent Request) . . . 13

9 Algorithm Analysis **13**
- 9.1 Advantages Over Ricart-Agrawala . . . 13

10 Conclusion **13**

Introduction

In a multiprogramming environment where multiple transactions can be executed simultaneously, it is highly important to control the **concurrency** of transactions. Concurrency control protocols ensure **atomicity**, **isolation**, and **serializability** of concurrent transactions.

The **Suzuki-Kasami Algorithm** is a mutual exclusion-based algorithm for distributed systems. It is a *token-based* algorithm — a site may enter its critical section only if it currently holds the unique circulating token.

Objectives

Objective

Understand the process of concurrency control by implementing the **Suzuki-Kasami Algorithm** in C, simulating multiple sites communicating over POSIX shared memory.

Tools / Software Requirements

- **Language:** C (ISO C11)
- **Compiler:** GCC (`gcc` / `cc`) on Linux / macOS
- **IPC:** POSIX shared memory (`shm_open`), semaphores, `pthreads`
- **Editor / IDE:** Visual Studio Code, Eclipse, or any text editor

Background Theory

Token-Based vs. Non-Token-Based Algorithms

Property	Token-Based	Non-Token-Based (e.g. R-A)
Ordering mechanism	Sequence numbers	Timestamps
Messages per CS entry	0–2	$2(N-1)$
Token loss risk	Yes (requires recovery)	No
Example	Suzuki-Kasami	Ricart-Agrawala

Data Structures

- **RN[1...N]** — Array maintained by every site S_i. $\mathrm{RN}_i[j]$ stores the largest sequence number received so far in a `REQUEST` message from site S_j.
- **LN[1...N]** — Array carried by the *token*. $\mathrm{LN}[j]$ is the sequence number of the request most recently executed by site S_j.
- **Q** — A FIFO queue stored in the token that records the IDs of sites currently waiting for the token.

Algorithm Steps

To Request / Enter the Critical Section

1. When site S_i wants to enter the CS and does *not* hold the token, it increments $\mathrm{RN}_i[i]$ and broadcasts `REQUEST`$(i,\ \mathrm{RN}_i[i])$ to all other sites.
2. Upon receiving `REQUEST`$(i,\ sn)$, site S_j sets
$$\mathrm{RN}_j[i] = \max\big(\mathrm{RN}_j[i],\ sn\big).$$
3. If S_j holds the token **and** $\mathrm{RN}_j[i] = \mathrm{LN}[i] + 1$, it sends the token to S_i.

To Execute the Critical Section

- S_i enters and executes the CS only after it has acquired the token.

To Release the Critical Section

After finishing execution, S_i:

1. Sets $\mathrm{LN}[i] = \mathrm{RN}_i[i]$.
2. For every site S_j whose ID is *not* in Q: if $\mathrm{RN}_i[j] = \mathrm{LN}[j] + 1$, append j to Q.
3. If Q is non-empty, pop the front ID and forward the token to that site.
4. If Q is empty, retain the token.

Implementation

The following C program simulates five sites using **POSIX shared memory** on a single machine. Each site runs in a separate terminal.

Complete Source Code — `suzuki_kasami.c`

```c
/*
 * suzuki_kasami.c
 *
 * Suzuki-Kasami distributed mutual exclusion algorithm
 * simulated on ONE machine using POSIX shared memory.
 *
 * Compile:  gcc -o sk suzuki_kasami.c -lrt -lpthread
 * Run     :  Open 5 separate terminals and execute:
 *                ./sk   (each terminal will ask for site number 1-5)
 *            Site 1 starts with the token.
 * Stop    :  Ctrl+C in any terminal.
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

```c
#include <pthread.h>
#include <semaphore.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <time.h>
#include <signal.h>

/*        Configuration                                                    */
#define N           5           /* total number of sites                   */
#define SHM_NAME    "/sk_shm"   /* shared memory segment name              */
#define QUEUE_CAP   (N + 1)     /* max queue size                          */

/*        Shared data structures                                           */

/* The token     lives in shared memory, "owned" by whoever holds it */
typedef struct {
    int  LN[N];                 /* LN[j]: last executed seq# of site j */
    int  queue[QUEUE_CAP];      /* FIFO queue of waiting site IDs      */
    int  q_head;
    int  q_tail;
    int  q_size;
    int  holder;                /* site index (0-based) holding token  */
} Token;

/* Per-site state     each entry is written only by site i */
typedef struct {
    int  RN[N];                 /* RN[j]: largest REQUEST seq# from j  */
    int  wants_cs;              /* flag: this site is requesting CS    */
    int  in_cs;                 /* flag: this site is in CS            */
} SiteState;

/* Full shared memory layout */
typedef struct {
    pthread_mutex_t  mutex;            /* protects all shared state  */
    pthread_cond_t   token_cond[N];    /* per-site condition variable*/
```

```
    Token             token;
    SiteState         site[N];
    int               initialized;      /* set to 1 after first init */
} SharedMem;

/*        Globals                                                    */
static SharedMem *shm  = NULL;
static int        me   = -1;    /* this site's 0-based index */
static volatile sig_atomic_t running = 1;

/*        Queue helpers (called with mutex held)                     */
static void q_push(Token *t, int id)
{
    if (t->q_size >= QUEUE_CAP) return;   /* should not happen */
    t->queue[t->q_tail] = id;
    t->q_tail = (t->q_tail + 1) % QUEUE_CAP;
    t->q_size++;
}

static int q_pop(Token *t)
{
    if (t->q_size == 0) return -1;
    int id = t->queue[t->q_head];
    t->q_head = (t->q_head + 1) % QUEUE_CAP;
    t->q_size--;
    return id;
}

static int q_contains(Token *t, int id)
{
    for (int k = 0; k < t->q_size; k++)
        if (t->queue[(t->q_head + k) % QUEUE_CAP] == id)
            return 1;
    return 0;
}

/*        Signal handler                                             */
static void handle_sigint(int sig)
{
    (void)sig;
    running = 0;
    printf("\n[Site %d] Shutting down...\n", me + 1);
}
```

```c

/*        Shared memory helpers
    */
static SharedMem *open_or_create_shm(void)
{
    int created = 0;
    int fd = shm_open(SHM_NAME, O_CREAT | O_EXCL | O_RDWR, 0666);
    if (fd >= 0) {
        created = 1;
        ftruncate(fd, sizeof(SharedMem));
    } else {
        fd = shm_open(SHM_NAME, O_RDWR, 0666);
        if (fd < 0) { perror("shm_open"); exit(1); }
    }

    SharedMem *s = (SharedMem *)mmap(NULL, sizeof(SharedMem),
                                     PROT_READ | PROT_WRITE,
                                     MAP_SHARED, fd, 0);
    close(fd);
    if (s == MAP_FAILED) { perror("mmap"); exit(1); }

    if (created) {
        memset(s, 0, sizeof(SharedMem));

        /* Process-shared mutex */
        pthread_mutexattr_t mattr;
        pthread_mutexattr_init(&mattr);
        pthread_mutexattr_setpshared(&mattr, PTHREAD_PROCESS_SHARED);
        pthread_mutex_init(&s->mutex, &mattr);
        pthread_mutexattr_destroy(&mattr);

        /* Process-shared condition variables */
        pthread_condattr_t cattr;
        pthread_condattr_init(&cattr);
        pthread_condattr_setpshared(&cattr, PTHREAD_PROCESS_SHARED);
        for (int i = 0; i < N; i++)
            pthread_cond_init(&s->token_cond[i], &cattr);
        pthread_condattr_destroy(&cattr);

        /* Token: site 0 (Site 1 to the user) starts with token */
        s->token.holder = 0;
        s->initialized  = 1;
        printf("[Shared memory] Created and initialized by Site %d\n",
               me + 1);
    }
```

```c
    return s;
}

/*        Core algorithm

    */

/*
 * request_cs():
 *   Site 'me' wants to enter the critical section.
 *   If we don't hold the token we broadcast REQUEST and wait.
 */
static void request_cs(void)
{
    pthread_mutex_lock(&shm->mutex);

    Token     *tok  = &shm->token;
    SiteState *self = &shm->site[me];

    /* Already holding token? Just mark and go. */
    if (tok->holder == me) {
        self->wants_cs = 0;
        self->in_cs    = 1;
        pthread_mutex_unlock(&shm->mutex);
        return;
    }

    /* Increment our sequence number and record the request */
    self->RN[me]++;
    int sn = self->RN[me];
    self->wants_cs = 1;

    printf("[Site %d] Broadcasting REQUEST(%d, sn=%d)\n",
           me + 1, me + 1, sn);

    /*
     * "Broadcast" to all other sites:
     * Each site Sj updates RNj[me] = max(RNj[me], sn).
     * If Sj holds the token and RNj[me] == LN[me]+1, it forwards.
     */
    for (int j = 0; j < N; j++) {
        if (j == me) continue;

        SiteState *sj = &shm->site[j];
        if (sj->RN[me] < sn)
            sj->RN[me] = sn;

        /* Does site j currently hold the token and should forward? */
        if (tok->holder == j &&
```

```c
            sj->RN[me] == tok->LN[me] + 1) {

            printf("[Site %d] Forwarding token to Site %d\n",
                   j + 1, me + 1);
            tok->holder = me;
            pthread_cond_signal(&shm->token_cond[me]);
            break;
        }
    }

    /* Wait until we receive the token */
    while (tok->holder != me)
        pthread_cond_wait(&shm->token_cond[me], &shm->mutex);

    self->wants_cs = 0;
    self->in_cs    = 1;
    pthread_mutex_unlock(&shm->mutex);
}

/*
 * release_cs():
 *   Site 'me' finishes its critical section and releases the
   token.
 */
static void release_cs(void)
{
    pthread_mutex_lock(&shm->mutex);

    Token     *tok  = &shm->token;
    SiteState *self = &shm->site[me];

    self->in_cs = 0;

    /* Step 1: update LN[me] */
    tok->LN[me] = self->RN[me];

    /* Step 2: enqueue sites with outstanding requests */
    for (int j = 0; j < N; j++) {
        if (j == me) continue;
        if (!q_contains(tok, j) &&
            shm->site[j].RN[j] == tok->LN[j] + 1) {
            q_push(tok, j);
            printf("[Site %d] Added Site %d to token queue\n",
                   me + 1, j + 1);
        }
    }

    /* Step 3: forward token or keep it */
    if (tok->q_size > 0) {
        int next = q_pop(tok);
        tok->holder = next;
```

```c
        printf("[Site %d] Releasing token -> Site %d\n",
               me + 1, next + 1);
        pthread_cond_signal(&shm->token_cond[next]);
    } else {
        printf("[Site %d] No pending requests; keeping token.\n",
               me + 1);
    }

    pthread_mutex_unlock(&shm->mutex);
}

/*        Critical section body
    */
static void execute_cs(void)
{
    printf("\n*** [Site %d] === ENTERING CRITICAL SECTION ===\n",
           me + 1);
    printf("*** [Site %d]     (Performing work for 2 seconds)\n",
           me + 1);
    sleep(2);   /* simulate work */
    printf("*** [Site %d] === EXITING  CRITICAL SECTION ===\n\n",
           me + 1);
}

/*        Print state helper
    */
static void print_state(void)
{
    pthread_mutex_lock(&shm->mutex);
    Token     *tok  = &shm->token;
    SiteState *self = &shm->site[me];

    printf("[Site %d] RN = [", me + 1);
    for (int i = 0; i < N; i++)
        printf("%d%s", self->RN[i], i < N-1 ? ", " : "");
    printf("]  LN = [");
    for (int i = 0; i < N; i++)
        printf("%d%s", tok->LN[i], i < N-1 ? ", " : "");
    printf("]  TokenHolder = Site %d\n", tok->holder + 1);
    pthread_mutex_unlock(&shm->mutex);
}

/*        main
    */
```

```c
int main(void)
{
    /* Read site number from user */
    int site_num;
    printf("Enter site number (1-%d): ", N);
    fflush(stdout);
    if (scanf("%d", &site_num) != 1 || site_num < 1 || site_num > N) {
        fprintf(stderr, "Invalid site number.\n");
        return 1;
    }
    me = site_num - 1;   /* convert to 0-based */

    signal(SIGINT, handle_sigint);

    /* Attach / create shared memory */
    shm = open_or_create_shm();

    /* Small delay to allow all sites to initialise */
    printf("[Site %d] Initialised. Waiting for other sites...\n",
           me + 1);
    sleep(2);

    printf("[Site %d] Press ENTER to request Critical Section"
           " (Ctrl+C to quit):\n", me + 1);

    while (running) {
        /* Wait for user keystroke */
        getchar();
        if (!running) break;

        printf("[Site %d] Requesting Critical Section...\n", me + 1);
        print_state();

        request_cs();      /* --- request + wait for token --- */
        execute_cs();      /* --- inside critical section  --- */
        release_cs();      /* --- release token            --- */

        print_state();
        printf("[Site %d] Press ENTER to request CS again"
               " (Ctrl+C to quit):\n", me + 1);
    }

    /* Cleanup shared memory if we are the last process */
    munmap(shm, sizeof(SharedMem));
```

```c
    shm_unlink(SHM_NAME);    /* harmless if another process does it too */
    printf("[Site %d] Exited cleanly.\n", me + 1);
    return 0;
}
```

Listing 1: suzuki_kasami.c — Full implementation

Compilation and Execution Guide

Compile

```
gcc -o sk suzuki_kasami.c -lrt -lpthread -std=c11 -Wall -Wextra
```

Listing 2: Compile command

Run (5 Separate Terminals)

Open **five terminal windows** on the same machine and run the binary in each:

```
./sk
```

Listing 3: Execution in each terminal

- Each terminal will prompt: `Enter site number (1-5):`
- Enter a *unique* number (1 through 5) in each terminal.
- After a 2-second init delay, each terminal shows:
  `Press ENTER to request Critical Section (Ctrl+C to quit):`
- **Site 1 starts with the token.**
- Press `ENTER` on any site to request entry into the CS.
- The program runs continuously until terminated with `Ctrl+C`.

Important Note on Shared Memory

Because all five processes share the *same* shared memory segment (`/sk_shm`), this simulation correctly models multiple nodes sending and receiving messages — all coordinated via the shared token structure. If the program crashes and leaves stale shared memory, remove it manually:

```
rm /dev/shm/sk_shm
```

Step-by-Step Execution Trace

The table below traces a sample 3-site execution (for clarity, $N = 3$).

Step	Event	RN_1	RN_2	RN_3	To
0	Initial state	[0,0,0]	[0,0,0]	[0,0,0]	LN
1	S2 requests CS (sn=1)	[0,1,0]	[0,1,0]	[0,1,0]	LN
2	S1 holds token, $RN_1[2]=1=LN[2]+1$; forwards	[0,1,0]	[0,1,0]	[0,1,0]	LN
3	S2 enters CS	[0,1,0]	[0,1,0]	[0,1,0]	H=
4	S3 requests CS (sn=1) while S2 in CS	[0,1,1]	[0,1,1]	[0,1,1]	H=
5	S2 exits CS: LN=[0,1,0]; checks S3: RN[3]=1=LN[3]+1; Q={S3}				H–
6	S3 enters CS				LN
7	S3 exits CS: LN=[0,1,1]; Q={}				To

Expected Output (Screenshots)

Terminal — Site 1 (Token Holder at Start)

```
Enter site number (1-5): 1
[Shared memory] Created and initialized by Site 1
[Site 1] Initialised. Waiting for other sites...
[Site 1] Press ENTER to request Critical Section (Ctrl+C to quit):

[Site 1] Requesting Critical Section...
[Site 1] RN = [0, 0, 0, 0, 0]  LN = [0, 0, 0, 0, 0]  TokenHolder = Site 1

*** [Site 1] === ENTERING CRITICAL SECTION ===
*** [Site 1]     (Performing work for 2 seconds)
*** [Site 1] === EXITING  CRITICAL SECTION ===

[Site 1] RN = [1, 0, 0, 0, 0]  LN = [1, 0, 0, 0, 0]  TokenHolder = Site 1
[Site 1] No pending requests; keeping token.
[Site 1] Press ENTER to request CS again (Ctrl+C to quit):
```

Terminal — Site 2 (Requests After Site 1)

```
Enter site number (1-5): 2
[Site 2] Initialised. Waiting for other sites...
[Site 2] Press ENTER to request Critical Section (Ctrl+C to quit):

[Site 2] Requesting Critical Section...
[Site 2] Broadcasting REQUEST(2, sn=1)
[Site 1] Forwarding token to Site 2
[Site 2] RN = [0, 1, 0, 0, 0]  LN = [0, 0, 0, 0, 0]  TokenHolder = Site 2

*** [Site 2] === ENTERING CRITICAL SECTION ===
*** [Site 2]     (Performing work for 2 seconds)
*** [Site 2] === EXITING  CRITICAL SECTION ===

[Site 2] RN = [0, 1, 0, 0, 0]  LN = [0, 1, 0, 0, 0]  TokenHolder = Site 2
[Site 2] No pending requests; keeping token.
[Site 2] Press ENTER to request CS again (Ctrl+C to quit):
```

Terminal — Site 3 (Concurrent Request)

```
Enter site number (1-5): 3
[Site 3] Initialised. Waiting for other sites...
[Site 3] Press ENTER to request Critical Section (Ctrl+C to quit):

[Site 3] Requesting Critical Section...
[Site 3] Broadcasting REQUEST(3, sn=1)
[Site 3] RN = [0, 0, 1, 0, 0]  LN = [0, 1, 0, 0, 0]  TokenHolder = Site 2
(Site 3 waiting for token -- Site 2 is in CS)
[Site 2] Added Site 3 to token queue
[Site 2] Releasing token -> Site 3

*** [Site 3] === ENTERING CRITICAL SECTION ===
*** [Site 3]     (Performing work for 2 seconds)
*** [Site 3] === EXITING  CRITICAL SECTION ===

[Site 3] RN = [0, 0, 1, 0, 0]  LN = [0, 1, 1, 0, 0]  TokenHolder = Site 3
```

Algorithm Analysis

Property	Value / Description
Message complexity	0 to $N-1$ REQUEST + 1 TOKEN per CS entry
Synchronization delay	At most $N-1$ message transfer delays
Mutual exclusion	Guaranteed (only one token exists)
Starvation freedom	Yes — FIFO queue in token prevents starvation
Deadlock freedom	Yes — token always progresses
Token loss	Requires a separate recovery protocol

Advantages Over Ricart-Agrawala

- Requires at most $N-1$ messages instead of $2(N-1)$.
- Does not rely on global clock synchronisation.
- Outstanding requests are tracked efficiently via the Q queue in the token.

Conclusion

This lab demonstrated the **Suzuki-Kasami token-based mutual exclusion algorithm** in a distributed setting simulated via POSIX shared memory on a single host.

Key takeaways:

- Only the site holding the *token* can enter the critical section, ensuring **mutual exclusion**.
- The **RN** array at each site and the **LN** array inside the token together distinguish old requests from new ones using **sequence numbers**.

- The **FIFO queue** Q inside the token ensures that all outstanding requests are eventually serviced, preventing **starvation**.
- The algorithm reduces message overhead significantly compared to Ricart-Agrawala by leveraging the token as a shared communication object.